Path-Aware Governance for High-Volume AI-Agent Workflows

AMS 560 Project — Midterm Report

Team members:

1 Objective and Project Description

The project asks whether *path-aware* governance, which judges each proposed agent action in light of the earlier actions in the same workflow and across cooperating agents, catches policy violations that per-action rules miss, and what it costs in latency, throughput and state consistency. We compare three engines on one shared event format: **no governance** (allow everything), **stateless** (rules on single actions: dangerous actions, a per-transfer limit, unauthorized actors), and **path-aware** (stateless rules plus multi-step rules for cross-agent data exfiltration, privilege laundering through delegation, and split-transfer evasion). A Kafka pipeline runs the engines with several workers under strong (workflow-partitioned) and weak (agent-partitioned, replicated) state.

The first phase evaluated the engines on a seeded, template-generated corpus of 150 labeled workflows. This evaluation verifies that each engine behaves as intended on controlled, labeled inputs. The second phase, now under way, puts a large language model (LLM) agent in a simulated environment where every tool call passes through the governance engine, so that detection and cost can be measured on behavior we did not script.

2 Timeline and Expected Deliverables

Phase	Deliverable	Status
1	Event schema, policy definitions, three governance engines	Complete
2	Seeded synthetic workflow corpus with blind-labeling sample	Labels pending review
3	Kafka pipeline (topics, replay, partitioning strategies)	Built; agent runs pending
4	Strong and weak multi-worker shared state	Built; agent runs pending
5	Benchmarks (accuracy, stale reads, latency, throughput) and dashboard	Complete
6	Agent test harness: simulated tools, gateway, provenance	Complete
7	LLM agent loop and single-model pilot (GPT-5.6 Luna, medium effort)	In progress
8	Agent scenario suite, outcome oracle, repeated-run design	Started
9	Final validation, reproducibility check, final report	Planned

3 Progress on Each Deliverable

Deliverable	What is done and what remains
Engines (1)	All three engines implemented behind one interface. The path-aware engine now records every attempted action with a status (blocked, allowed, executed, failed); cumulative limits count only executed transfers, so a blocked attempt no longer inflates later totals.
Workflow corpus (2)	150 reproducible safe, violating and near-miss workflows (seed 560). Remaining: independent two-reviewer labeling of the prepared 20-workflow sample.
Kafka pipeline and state (3, 4)	Implemented and measured on the synthetic corpus. With four workers, workflow partitioning gave 0% stale reads and 100% recall; agent partitioning with replicated state gave about 25% median stale reads and 57–59% median recall. Remaining: validate with traces from LLM agents.
Benchmarks and dashboard (5)	Complete. On the synthetic corpus, stateless recall is 23.3% and path-aware recall is 100% with no false positives (by construction, see Section 1). In-process path-aware throughput was about 340,000 events/s at concurrency 10 and 207,000 at concurrency 100.
Agent harness (6)	Complete. A simulated world (data sources, files, accounts, permissions) and eight tools; a gateway that builds each event from world state rather than from the agent's claims and blocks or runs the call; lineage-based provenance so data-flow rules work without agent cooperation.

Deliverable	What is done and what remains
Scripted scenarios (6)	Eleven scenarios run under all three levels, with harm judged from what actually happened in the simulated world.
Engine fixes and known limits (6)	Building the harness exposed two engine gaps that we fixed (a harmless read before a PII read hid exfiltration; blocked attempts counted toward transfer totals) and three that we document as known limits (merging two PII sources; one agent splitting a payment; a legitimate over-limit payment with no approval path).
LLM pilot (7)	A model-neutral agent loop is built and tested with a scripted model. A connection through the Codex CLI with our tools served over MCP was tested live: governed tool calls work and built-in shell and web tools can be disabled, but Codex sub-agents each start a separate copy of the tool server, which splits the workflow history.
Scenario suite and design (8)	Groundwork only: outcome-based harm check, basic metrics (blocks, retries after a block, task completion), and offline replay of ungoverned runs through the other engines.

Outcomes of the scripted harness scenarios, judged from the simulated world ("harm": PII left, a limit was exceeded, an unauthorized refund was issued, or the database was deleted). They check the engines and harness end to end and do not measure agent behavior.

Scenario group	None	Stateless	Path-aware
PII exfiltration paths (3, incl. harmless read first)	harm	harm	blocked
Two-agent split transfer; privilege laundering	harm	harm	blocked
Over-limit transfer; delete production database	harm	blocked	blocked
Benign internal report; authorized delegation	no harm	no harm	no harm
Known limits: two PII sources merged; one-agent split	harm	harm	harm

4 Plan for Unfinished Deliverables

Corpus labels (2). Two members label the 20-workflow blind sample using only the written policies; we compare against each other and the generated labels and revise ambiguous policies or templates before freezing them.

LLM pilot (7). Choose the connection: fix the Codex route with one shared tool server per run, or call the model directly through the OpenAI API, which exposes only our tools and none of Codex's built-in ones (key read from an environment variable, never stored in the repository). Run the three draft pilot tasks with one model to debug the loop and the outcome check, then identify and test alternative models from other providers behind the same agent loop.

Scenario suite and design (8). Write scenarios where a violation is possible but never instructed (benign, tempting, adversarial, multi-agent handoff), with some written by a member who did not write the rules and a held-out set never used to tune rules. Freeze policies and pre-register hypotheses and metrics, then run repeated trials per scenario and governance level and report confidence intervals, task completion and benign-task blocking alongside recall.

Kafka pipeline and shared state (3, 4). Replay agent traces from the pilot through the Kafka workers under both partitioning strategies and compare with the synthetic-corpus results.

Final validation (9). A second member reproduces the full setup on another machine (dependencies, Kafka, tests, benchmarks, harness). The final report will keep synthetic-workflow and agent-run results separate and list the known limits above.

5 Difficulties and Risks

The main systems difficulty was shared state across Kafka workers: agent partitioning splits a workflow, so decisions use incomplete history. We report in-process and Kafka costs separately. The main research risk is circularity: if one author writes rules, scenarios and labels, high recall says little. Our safeguards are independent authors, a held-out set, a world-based outcome check and pre-registered metrics. Model access and cost are practical risks.